Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.02.02**

Тема: **Проектирование клиентской архитектуры и OpenAPI-спецификации шлюза**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.
Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОПИСАНИЕ ЗАДАНИЯ.....4
2 СПЕЦИФИКАЦИЯ OPENAPI.....5
3 КЛИЕНТ: СТРУКТУРА, КЛАСС И ТЕСТ.....6
ЗАКЛЮЧЕНИЕ.....8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....9

ВВЕДЕНИЕ

Цель работы: освоить подход contract-first [1] – сначала описать контракт взаимодействия клиента и шлюза в формате OpenAPI [2], а затем строить клиентское приложение на его основе, разделив интерфейс и сетевой слой.

Работа выполнена по варианту 3: АИС «Электронный магазин / Маркетплейс кампуса». Шлюз обслуживает каталог сувенирной продукции университета и приём заказов.

1 ОПИСАНИЕ ЗАДАНИЯ

Требуется: составить спецификацию openapi.yaml версии 3.0.3 для двух связанных эндпоинтов GET /catalog/items (список товаров) и POST /orders/checkout (оформление заказа) с общими схемами в components/schemas; создать в Visual Studio проект клиента со структурой папок UI, Services, Models; написать класс клиента CampusApiClient и базовый тест десериализации ответа.

Таблица 1 – Эндпоинты шлюза

Метод и путь	**Назначение**	**Ответы**
GET /catalog/items	Список товаров с фильтром category и страницами	200, 400, 500
POST /orders/checkout	Оформление заказа по списку товаров	201, 400, 409

Эндпоинты связаны: идентификатор itemId в теле заказа берётся из поля id товара, полученного из каталога.

2 СПЕЦИФИКАЦИЯ OPENAPI

Файл openapi.yaml проверен валидатором openapi-spec-validator: ошибок нет, найдено два пути и шесть схем (CatalogItem, CatalogItemList, OrderLine, CheckoutRequest, CheckoutResponse, Error). Общие структуры вынесены в components/schemas и подключаются через $ref, поэтому не дублируются. Полный файл приложен к работе.

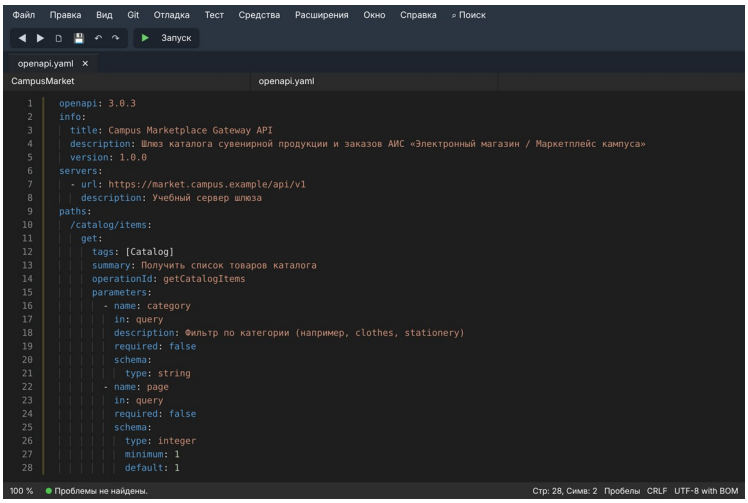


Рисунок 1 – Заголовок спецификации и эндпоинт GET /catalog/items

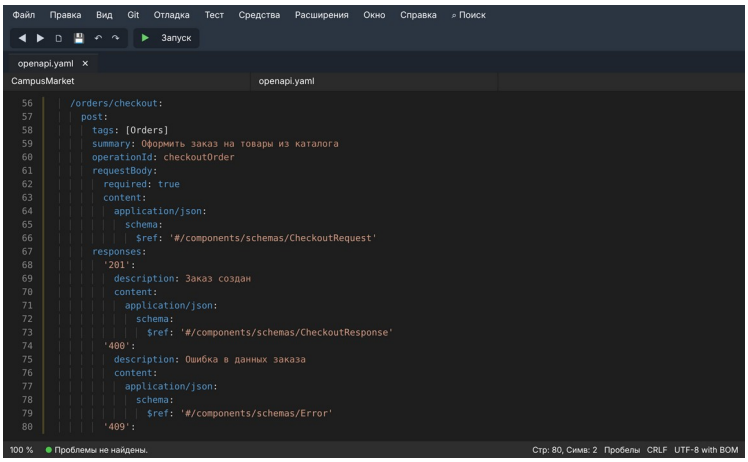


Рисунок 2 – Эндпоинт POST /orders/checkout и его ответы

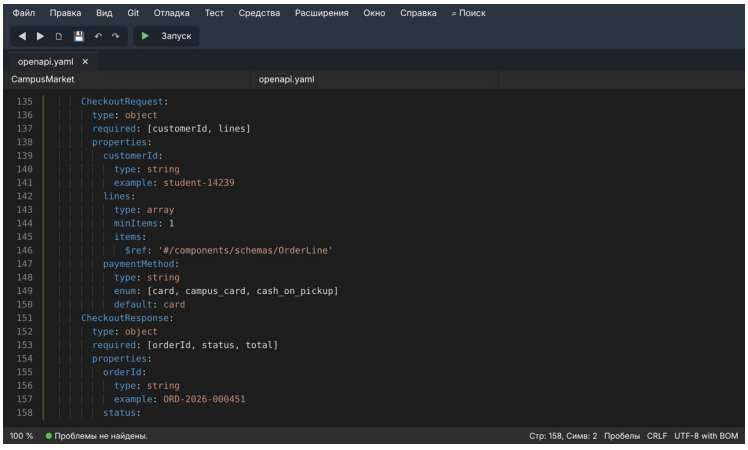


Рисунок 3 – Схемы OrderLine, CheckoutRequest и CheckoutResponse

3 КЛИЕНТ: СТРУКТУРА, КЛАСС И ТЕСТ

3.1 Структура папок

Клиент разделён по принципу разделения ответственности [3]: папка Models содержит классы данных по схемам контракта, Services – сетевой слой с HTTP и JSON, UI – вывод и ввод. Интерфейс уровня UI знает только ICampusApiClient и ничего не знает о HttpClient.

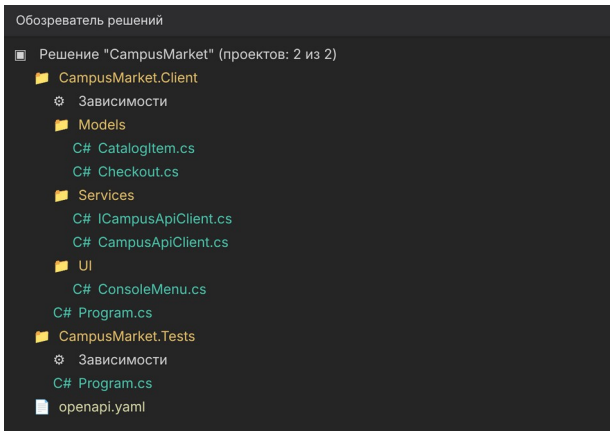


Рисунок 4 – Структура решения в обозревателе решений Visual Studio

3.2 Класс CampusApiClient и тест

Класс CampusApiClient принимает HttpClient через конструктор [4], выполняет запросы, разбирает JSON средствами System.Text.Json [5] и превращает ответы с ошибками в исключение CampusApiException с кодом из схемы Error.

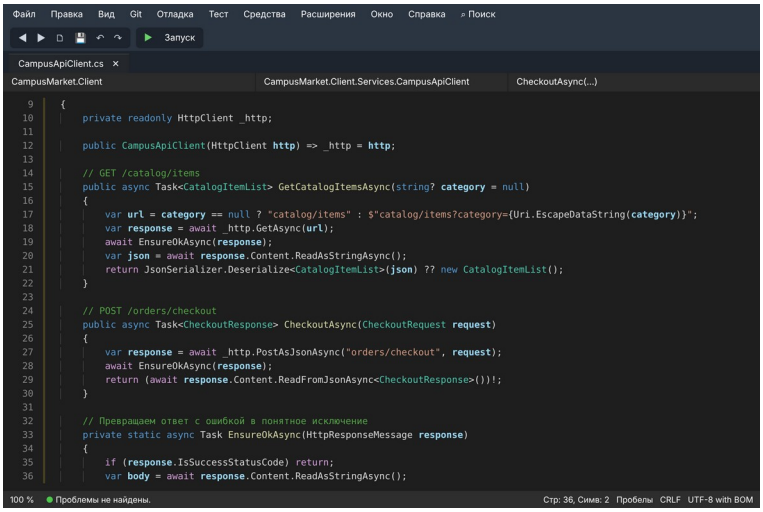


Рисунок 5 – Методы GetCatalogItemsAsync, CheckoutAsync и обработка ошибок

Базовый тест подменяет HTTP-обработчик заглушкой, которая возвращает заранее заданный JSON, поэтому сеть не нужна. Проверяются

разбор списка товаров, полей товара, ответа оформления заказа и преобразование ответа 409 в исключение.

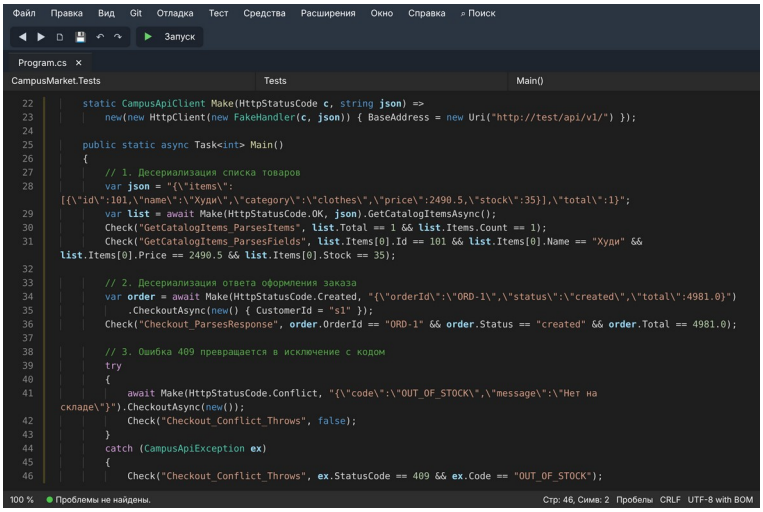


Рисунок 6 – Тесты десериализации с заглушкой FakeHandler

3.3 Результат запуска

Все четыре проверки пройдены: разбор списка товаров, полей товара и ответа оформления заказа выполняется верно, а ответ 409 превращается в исключение CampusApiException с кодом OUT_OF_STOCK.

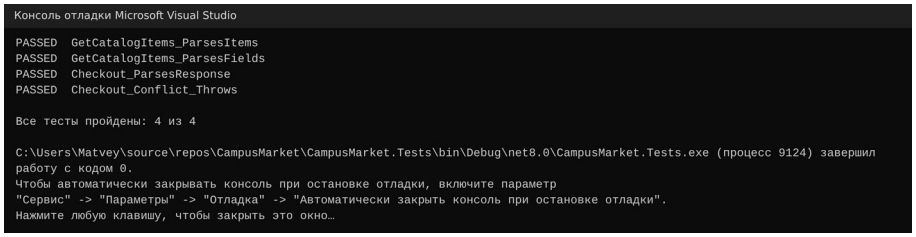


Рисунок 7 – Результат выполнения тестов

ЗАКЛЮЧЕНИЕ

Составлена и проверена спецификация OpenAPI 3.0.3 для шлюза маркетплейса кампуса, на её основе создан клиент со структурой Models, Services, UI и протестирована десериализация ответов.

Разделение интерфейса и сетевого слоя даёт несколько преимуществ: интерфейс можно менять, не затрагивая работу с сетью; сетевой слой тестируется без сервера и без интерфейса; изменение контракта правится в одном месте; клиент легко подключить к другому интерфейсу, например к WPF вместо консоли. Подход contract-first позволяет начать разработку клиента до готовности шлюза.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Swagger. Design First vs Code First [Электронный ресурс]. – URL: https://swagger.io/blog/code-first-vs-design-first-api/ (дата обращения: 30.10.2026).

2. OpenAPI Initiative. OpenAPI Specification v3.0.3 [Электронный ресурс]. – URL: https://spec.openapis.org/oas/v3.0.3 (дата обращения: 30.10.2026).

3. Microsoft Learn. Common web application architectures [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures (дата обращения: 30.10.2026).

4. Microsoft Learn. Make HTTP requests with the HttpClient class [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/http/httpclient (дата обращения: 30.10.2026).

5. Microsoft Learn. System.Text.Json overview [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview (дата обращения: 30.10.2026).